Development of a Modular ROS2 Software Framework for Autonomous Vehicles

Sugyeong Kim[1)]·Yeonseo Woo[2)] ·Kyunam Kim[1)]
[1)]*School of Mechanical Engineering, SungKyunKwan University, 35, Seobu-ro 2126-gil, Jang-an-gu, Korea*
[2)]*Department of Computer Science and Engineering, SungKyunKwan University, 35, Seobu-ro 2126-gil, Jang-an-gu, Korea*

Key words: Autonomous Driving, ROS2, Modular Software Framework, Multi-Sensor Integration, Perception, Decision-Making

Abstract: Autonomous driving requires the integration of multiple sensors, perception algorithms, and decision-making modules in a reliable and scalable framework. This study presents the development of a modular software framework based on Robot Operating System 2 (ROS2) for autonomous vehicles. The proposed framework is composed of independent packages, including camera, LiDAR, decision-making, and vehicle communication modules. Each module is designed to operate independently while exchanging information through ROS2 topics, which improves system scalability and maintainability. The framework was implemented and tested on a competition-scale autonomous vehicle platform, corresponding to roughly one-fifth of the dimensions of a real vehicle. Experimental results from both simulation and real-world scenarios demonstrate that the modular design facilitates efficient integration of perception and decision-making functions while ensuring flexible adaptation to different sensors and algorithms. This work highlights the effectiveness of a ROS2-based modular architecture for autonomous driving applications and provides a foundation for future research on scalable autonomous vehicle systems.

1. INTRODUCTION

Autonomous driving has emerged as a core technology in future mobility, attracting considerable attention from both academia and industry. Recent studies highlight the importance of modular and scalable software frameworks that can integrate multiple sensors and decision-making algorithms into a unified system.

This paper presents the work developed for the 3rd Sungkyunkwan University Autonomous Driving Competition. The competition consisted of three categories (general driving, mission driving, and perpendicular parking) each requiring different functionalities such as lane keeping, traffic light recognition, and obstacle avoidance. These diverse requirements underscored the necessity of a modular approach, which enabled the system to selectively combine only the necessary modules for each task. As a result, unnecessary computations were reduced, flexible reconfiguration according to task demands became possible, and the efficiency of experimentation and debugging was significantly improved.

Therefore, the objective of this study is to design and implement a modular autonomous driving software framework based on ROS2. The proposed framework integrates perception, decision-making, and control processes while ensuring modularity and scalability, and its effectiveness was validated through real-world application on a competition-scale autonomous vehicle platform.

* Kyu Nam Kim, E-mail: kyunam.kim@skku.edu

2. SYSTEM ARCHITECTURE OVERVIEW

2.1 Full System Overview Diagram

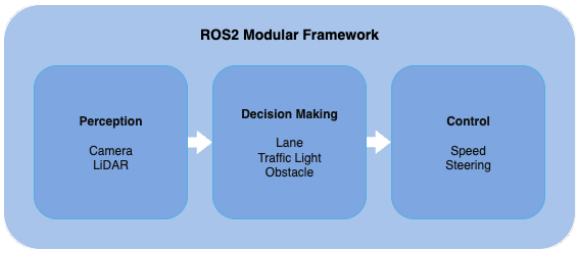


Fig. 1 ROS2 Modular Framework

The ROS2 Modular Framework in this study was modularized based on the three fundamental elements of autonomous driving: perception, decision making, and control.

The Perception module continuously perceives the surrounding environment in real time through camera and LiDAR sensors, and the collected data is forwarded to the Decision-Making module, which performs critical tasks such as lane detection, traffic light color identification, and obstacle recognition. Subsequently, the Control module precisely adjusts the vehicle's actual movements, managing speed, and steering control based on the decisions made.

2.2 Hardware Overview

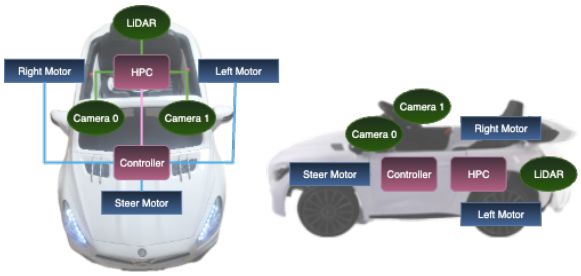


Fig. 2 Hardware configuration

The hardware configuration of the autonomous vehicle is illustrated in Fig. 2, and the specifications of the hardware components used are presented in Table 1.

Table 1 Hardware Specifications

Computing Device	HPC	ASUS TUF Gaming a15
	Controller	Arduino Uno
Sensor	Camera	Logitech C920
	LiDAR	RPLiDAR A1M8-R6
Motor Driver		SZH-GNP521

2.3 Software Overview

Table 2 ROS2 Package List

Package Name	Function
camera_pkg	Image capture, Lane detection
lidar_pkg	LiDAR data acquisition
decision_making_pkg	Sensor fusion, Driving decisions
control_pkg	Control command generation, Arduino transmission
interface_pkg	Message definition

The list of ROS2 packages used is presented in Table 2. The ‘camera_pkg’ handles image capture and lane detection, while the ‘lidar_pkg’ collects LiDAR data. The ‘decision_making_pkg’ performs sensor fusion and driving decisions. The ‘control_pkg’ is responsible for control command generation and communication with the Arduino. The ‘interface_pkg’ manages message definitions.

3. TYPES OF PACKAGES

3.1 Camera Package

The camera package is designed to handle to overall perception tasks of the camera module. It is responsible for capturing raw images and processing them for various perception functions, including lane detection, obstacle recognition, and traffic light color identification.

3.1.1 Image Publisher

The image publisher node provides a unified image source for downstream perception by exposing a parameterized interface to three data modalities: live camera, image directory, and video file. At run time, the *data_source* $\in$ *{camera, image, video}* parameter selects the input, and the publishing period is controlled by timer period, enabling precise frame-rate adjustment independent of the capture backend. For each cycle, frames are resized to 640×480, converted to *sensor_msgs/Image* via CvBridge, time-stamped, and published on the configurable topic.

3.1.2 YOLO Segmentation

The YOLOv11 segmentation node performs real-time object detection and segmentation on incoming image streams [1]. It subscribes to the raw image topic and processes each frame through a pre-trained YOLO-based segmentation model to extract pixel-level class masks. Unlike detection-oriented approaches that provide only bounding boxes, this node outputs dense segmentation results, enabling downstream modules such as lane detection, obstacle detection, and traffic light recognition to access detailed spatial information of objects within the scene. The segmentation results are published to a dedicated topic and serve as the foundation for subsequent perception tasks in the system.

3.1.3 Lane Detection

The Lane Detection node extracts objects labeled as *lane1* and *lane2* from the segmentation results and reconstructs them into a bird’s-eye view (BEV) representation. The BEV transformation removes perspective distortion and provides a planar coordinate system aligned with the vehicle’s frame, making the data directly suitable for control computation. The transformed lane data are further refined through filtering steps such as morphological operations, validity checks, and erosion to eliminate noise and ensure lane continuity.

The processed lane information is published in two forms. First, a dedicated topic for RViz visualization allows developers to intuitively verify the perception results. Second, a structured topic containing lane slope, lane ID, and vehicle position is published for decision-making and control modules. This structured information is directly utilized for lane keeping and lane-change decisions, while the separation of visualization and control data improves

modularity and debugging efficiency within the autonomous driving pipeline.

(a) (b)

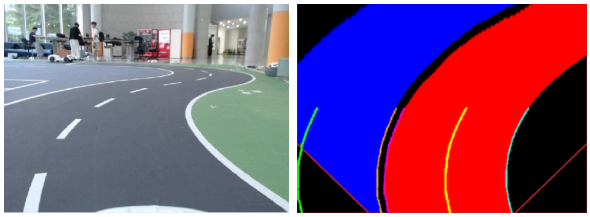


Fig. 3 Lane detection results: (a) raw image, (b) processed output

3.1.4 Obstacle Detection

The obstacle detection node integrates raw image frames with segmentation results labeled as *car* to identify frontal obstacles. The two input streams are synchronized, and a central region-of-interest is defined across the middle half of the image width. From the detection results, only objects labeled as car are considered. For each instance, the bounding-box center and area are computed, and an obstacle is confirmed if the object's center lies within the central region. To support interpretability, a mask image is generated by overlaying bounding boxes on the original frame, where objects inside the central region are highlighted in red and those outside in blue.

The node outputs two topics: a status message (obstacle_result) that indicates the presence of an obstacle along with its estimated bounding-box area, and a visualization mask (obstacle_mask) that provides an interpretable representation of the detection for downstream modules and qualitative evaluation.

(a) (b)



Fig. 4 Obstacle detection results: (a) raw image, (b) processed output

3.1.5 Traffic Light Recognition

The traffic light recognition node fuses raw image frames with segmentation results to infer the current signal state. The two inputs are time-synchronized, and for each detection labeled *traffic_light*, the node extracts the corresponding image region (ROI) from the bounding box. The ROI is converted to HSV, and color masks are applied using predefined ranges for red, yellow, and green; the nonzero pixel counts from these masks serve as a voting metric. The dominant color is selected by argmax over the three areas (falling back to *unknown* when all are zero), and the associated bounding-box area is recorded as a coarse proxy for signal salience. An interpretable overlay is then produced by alpha-blending a filled rectangle over each traffic-light ROI, colored according to the inferred state (red/yellow/green). The node publishes a concise status string reporting detection, area, and color as *traffic_light_result*, and a visualization frame with the overlay as *traffic_light_mask*.

(a) (b)

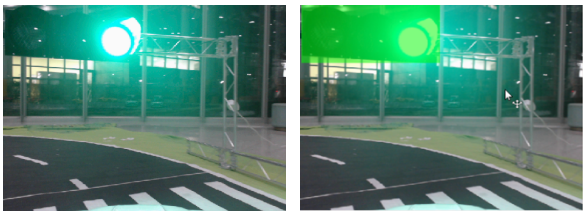


Fig. 5 Traffic Light Recognition results: (a) raw image, (b) processed output

3.2 LiDAR Package

The LiDAR package processes raw point cloud and scan data to support environmental perception. It includes a publisher node that interfaces with the LiDAR sensor through a serial connection and provides raw scan data to the system. Parameters such as serial port, baud rate, and frame ID can be adjusted to match the hardware setup, enabling straightforward integration into downstream perception modules.

3.3 Decision Making Package

The decision-making package integrates perception results from the camera and LiDAR packages to generate appropriate driving strategies. It interprets lane positions, traffic light states, and obstacle data to decide whether the vehicle should maintain its lane, stop, or perform avoidance maneuvers. This package outputs high-level driving commands in the form of velocity and steering messages, which are then transmitted to the control module.

To handle diverse driving environments, the decision-making package is structured into three specialized nodes. The *motion* node is responsible for general road driving, where it focuses on lane keeping and smooth maneuvering. The *motion mission* node addresses more complex situations that involve both obstacles and traffic lights, ensuring safe obstacle avoidance and compliance with traffic signals. Finally, the *motion parking* node is designed for parking lot scenarios, enabling precise low-speed maneuvers required for accurate parking. By dividing the decision-making process into these context-specific nodes, the framework enhances flexibility and robustness, allowing the vehicle to adapt its behavior according to different operating environments.

3.3.1 Motion

In this node, steering is computed in two modes. During lane changes, the node issues a fixed command $s = \pm 7$ toward the target lane with symmetric wheel speeds (*where*

s denotes the steering command of the vehicle, represented as an integer ranging from –10 to 10) and exits the mode once the reported lane equals the target and $|vehicle_position_x| \leq 100\,px$. During lane keeping, heading $\theta = steering_angle$ and lateral offset $x = vehicle_position_x$ are combined to form

$$s = clip((\frac{\theta}{50}) \times 10 \cdot k_\theta - xk_x, -10, 10)$$

With lane-dependent gains (default $k_\theta = 0.7,\ k_x = 0.05$). The negative sign on the offset term drives the vehicle back to center ($x > 0 \Rightarrow s < 0$ leftward, $x < 0 \Rightarrow s > 0$ rightward). The final integer s and wheel speeds are published each cycle as *MotionCommand* ∈ {left_speed, right_speed, steering}.

3.3.2 Motion Mission

The Motion Mission node integrates *Lane_info*, *traffic_light_result*, *obstacle_result*, and *LiDAR scan* data to determine high-level driving behaviors. By default, the vehicle follows the current lane using the information provided by *Lane_info*. When obstacles are detected, the node updates the target lane accordingly, and during a lane-change maneuver, steering is held constant to ensure a stable transition.

The response strategy differs depending on the type of obstacle. For static obstacles, the vehicle immediately performs a lane change to avoid collision. For dynamic obstacles, the node evaluates the direction and velocity of the obstacle and selects an appropriate avoidance maneuver based on the surrounding context.

In addition, *traffic_light_result* is used to recognize traffic signals. When a red light is detected, the vehicle stops, while for other signals it continues driving according to the given situation. Finally, the computed driving decision is output as *MotionCommand* ∈ {left_speed, right_speed, steering}, which is published as a topic to be consumed by downstream control modules.

3.3.3 Motion Parking

The Motion Parking node relies solely on *LiDAR scan* data to carry out the entire perpendicular parking process. Using LiDAR information, the node first recognizes an available parking slot and then initiates a sequence of actions including reverse entry, posture adjustment, and exit.

The algorithm is structured as a state-based process: once a slot is identified, the node commands the vehicle to move backward into the space, adjusting steering as necessary to align properly. After entering the slot, additional posture correction is performed to ensure the vehicle is positioned accurately within the boundaries. When parking is complete, the node is able to initiate an exit maneuver, guiding the vehicle out of the slot and back into its lane.

Throughout this process, the node continuously publishes *MotionCommand* ∈ {left_speed, right_speed, steering}, which provides the high-level control inputs required by the controller (Arduino). By using only LiDAR data for slot detection and positioning, the node is able to complete the full parking cycle autonomously in a modular and sensor-focused manner.

3.4 Control Package

The control package is responsible for delivering high-level driving commands to the Arduino, including speed of wheel and steering targets determined by the decision-making package. These commands represent abstract driving instructions, such as desired speed and steering angle, rather than direct hardware signals. The Arduino then translates these high-level commands into low-level control signals, such as PWM outputs and motor driver inputs, to operate the actuators. This design separates high-level decision making from low-level actuation, improving modularity and system flexibility.

In addition, the control package contains a keyboard control node, which enables manual driving by sending high-level commands generated from keyboard inputs.

3.4.1 Control

The control node bridges high-level driving targets to the Arduino over a lightweight serial link. It subscribes to MotionCommand and, on each callback, streams an ASCII frame to /dev/ttyACM0 at 9600 baud using the fixed schema.

The Arduino interprets these fields to generate low-level actuation (servo angle and motor PWM), preserving a clean separation between decision making and hardware control. A background keyboard loop provides an operator arm/disarm toggle: pressing the space bar switches between MOTION mode (forward incoming commands verbatim) and a fail-safe ZERO mode that enforces speed = 0, steering = 0 regardless of upstream activity, enabling immediate stop and safe re-arming without restarting node.

3.4.2 Keyboard Control

This node enables operator-in-the-loop teleoperation expressly for YOLO training dataset collection. It transmits steering and symmetric wheel-speed commands as lightweight ASCII frames over a serial link to the embedded controller, while simultaneously sampling frames from the onboard camera. A dedicated capture key stores the current frame to local storage for later labeling; other keys adjust speed and steering in bounded increments with predefined limits. All transmissions are logged for traceability. By combining manual driving with synchronized image capture, the node streamlines collection of diverse, task-relevant visuals for subsequent YOLO training without relying on automated perception in the loop.

3.5 Interface Package

The interface package defines standardized message schemas and topic interfaces that connect perception, decision-making, and control modules. By fixing field names, data types, and units, it ensures consistent data exchange across packages and enables drop-in replacement of algorithms without modifying downstream nodes. This

design improves modularity and facilitates easier testing, logging, and debugging throughout the system.

As an example, the package provides the *LaneInfo* message, which aggregates lane-related information for decision-making. It includes the steering angle of the lane centerline, the current lane number, and the vehicle's relative position. The definition is as follows:

```
# LaneInfo.msg

int32 steering_angle

int32 lane_num

int32 vehicle_position_x
```

Fig. 6 Definition of the LaneInfo.msg

This message is published by the lane detection module and subscribed to by the decision-making package, allowing lane information to be shared in a consistent and structured format.

4. IMPLEMENTATION

4.1 Development Environment

The system is implemented on Ubuntu 22.04 with ROS2 Humble. To ensure reproducibility of both the development and runtime environment, the Python 3.10/rclpy versions are fixed, and package dependencies are managed through ament metadata. The workspace follows a modular structure where each package is separated by functionality, and commonly used message types are placed in a dedicated package to reduce inter-module coupling and improve maintainability.

4.2 Launch File

The launch files serve to combine pre-developed nodes into comprehensive systems tailored to each competition task. By selectively grouping perception, decision-making, and control modules, the framework ensures flexibility and allows the entire system to be executed with a single command. In addition, parameters for each node can be specified directly within the launch files—for example, selecting the data source, assigning camera indices, defining topic names, or adjusting execution rates. This design eliminates the need to modify individual node files, enabling the system to be easily reconfigured for different scenarios without altering the underlying source code.

4.2.1 General Driving

General Driving launch file integrates camera, YOLO segmentation, lane detection, traffic light recognition, and motion nodes. Each node is configured with parameters such as data source, device, and topics for publishing or subscribing.

Table 3 General Driving Node List

Node	Key Parameter	Description
Image publisher0	Data source, cam_num, pub_topic	Select input source (camera/video/image), camera index, and publishing topic
Image publisher1		
YOLO Segmentation 0	Device, model_path, threshold	Defines computation device, model weights path, and detection confidence threshold
YOLO Segmentation 1		
Lane Detection	Image, detection	Subscribes to raw image and detection results for lane extraction
Traffic Light Recognition	-	Infers traffic light state (red/yellow/green) from segmentation results
Motion	-	Generates high-level driving commands (steering, velocity)

4.2.2 Mission Driving

Mission Driving launch file integrates dual camera inputs, YOLO segmentation, LiDAR, lane detection, traffic light recognition, obstacle detection, and motion mission nodes.

Table 4 Mission Driving Node List

Node	Key Parameter	Description
Image publisher0	data source, cam_num, pub_topic	Select input source (camera/video/image), camera index, and publishing topic
Image publisher1		
YOLO Segmentation 0	device, model_path, threshold	Defines computation device, model weights path, and detection confidence threshold
YOLO Segmentation 1		
TF	Parent frame, child frame, x y z roll pitch yaw	Publishes a static transform between *base_link* and *laser_frame*
Lidar	serial_port, serial_baudrate, frame_id, inverted, angle_compensate	Publishes LiDAR scan data with configured serial settings and frame ID
Lane Detection	image, detection	Subscribes to raw image and detection results for lane extraction
Obstacle Detection	-	Combines image frames and detection results to identify and publish frontal obstacles
Traffic Light Recognition	-	Infers traffic light state (red/yellow/green) from segmentation results
Motion Mission	-	Fuses multi-sensor perception and generates high-level driving commands (steering, velocity)

4.2.3 Perpendicular Parking

Perpendicular Parking launch file integrates LiDAR, a static transform publisher, and the motion parking node. Using only LiDAR scan data, the system performs slot recognition, reverse entry, posture adjustment, and exit maneuvers, while the motion parking node generates MotionCommand outputs for control.

Table 5 Perpendicular Parking Node List

Node	Key Parameter	Description
TF	Parent frame, child frame, x y z roll pitch yaw	Publishes a static transform between *base_link* and *laser_frame*
Lidar	serial_port, serial_baudrate, frame_id, inverted, angle_compensate	Publishes LiDAR scan data with configured serial settings and frame ID
Motion Parking	-	Executes the full parking cycle and publishes MotionCommand for control

4.3 Modularity & Scalability

The modular design of the framework is reflected in how perception and decision-making nodes are composed depending on the competition task. Two image publisher nodes are used to provide raw image streams, and each is paired with a YOLO segmentation node to produce object-level detections. This structure allows multiple perception pipelines to operate independently under separate namespaces.

Based on the task, only the required perception modules are activated. For general driving, lane detection is combined with the motion node to enable lane following. Mission driving integrates lane detection, traffic light recognition, and obstacle detection, providing the necessary perception for signal compliance and obstacle avoidance. Perpendicular parking uses only LiDAR data with the motion parking node, without camera-based modules.

Decision-making is similarly modular. The motion nodes are divided into general driving, mission driving, and parking variants, each tailored to its respective scenario. By keeping publishers, perception modules, and motion nodes independent, the system can be reconfigured for each task without modifying the underlying node implementations, ensuring both modularity and scalability across competition categories.

4.4 Example Workflow

The workflows corresponding to each scenario are presented as follows.

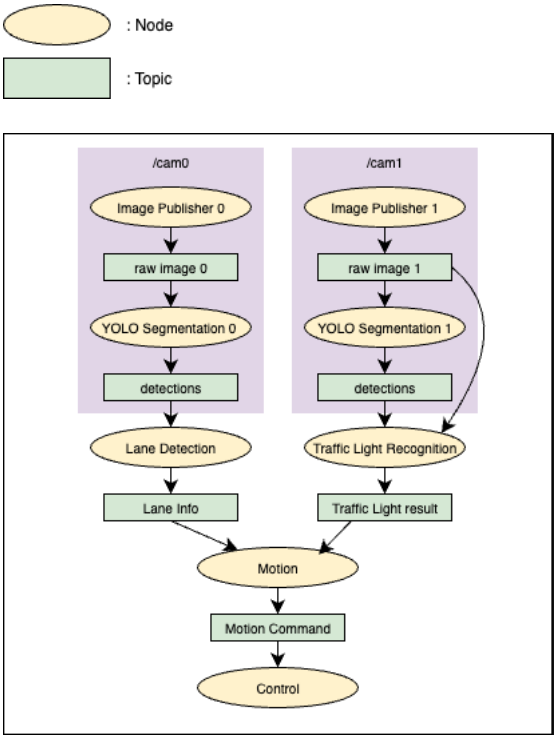


Fig. 7 General Driving Workflow

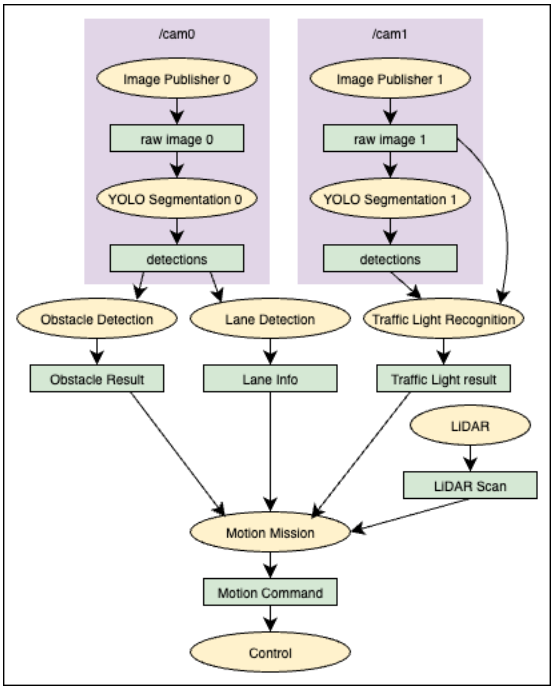


Fig. 8 Mission Driving Workflow

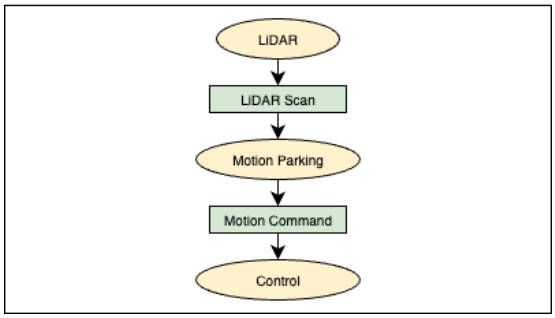
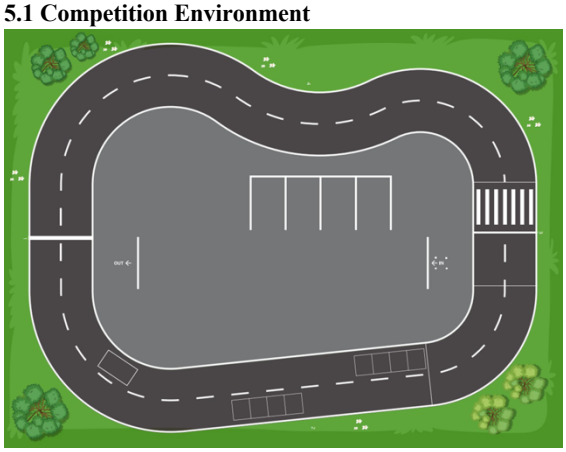


Fig. 9 Perpendicular Parking Workflow

5. VALIDATION AND RESULT

5.1 Competition Environment


Fig. 10 Layout of the competition track

The track was constructed with a road width of 850 mm. The lane markings, including both solid and dashed lines, are 50 mm wide, while the start line is 100 mm wide. The crosswalk measures 1000 mm by 100 mm, and each parking space is 950 mm by 1500 mm.

5.2 Driving Conditions

The event-specific rules are as follows:

5.2.1 General Driving

The vehicle started from the start line upon the referee's signal and completed two laps in a counterclockwise direction, with the total time recorded from departure to the final crossing of the start line. A mandatory lane-change was included in the crosswalk section: after 0.5 lap the vehicle switched from the outer to the inner lane, and after 1.5 laps it switched back to the outer lane.

5.2.2 Mission Driving

The obstacle-avoidance time was measured from the referee's signal to the moment the vehicle's front passed the success line. Three obstacles were included: a fixed one in the inner lane, a fixed one in the outer lane of a straight section, and a moving one in the inner lane. Crossing the center line in this section did not incur penalties, and passing the exit line marked mission success. In the crosswalk scenario, the vehicle had to stop before the crosswalk and restart only when the traffic light turned green, with the signal manually changed by the referee after a random delay. Successful mission completion was recorded when the vehicle crossed the designated line with its front.

5.2.3 Perpendicular Parking

The vehicle was required to park between two vehicles positioned side by side. Starting from a designated 3 cm square aligned with the front-right wheel, the vehicle had to enter the space in reverse, ensuring that both front and rear wheels were entirely within the marked area. To succeed, the vehicle had to stop for at least two seconds inside the space before exiting, and completion was recorded when the front wheels crossed the opposite OUT line.

5.3 Result

In the general driving task, the vehicle successfully completed 20 training laps without any lane violations, and in the official competition it achieved a lap time of 1 minute 32.67 seconds. In the mission driving task, static obstacles were consistently avoided during training, while dynamic obstacles were successfully avoided with a probability of about 50%; traffic light detection and stopping were achieved with 100% success. In the official competition, all mission tasks were completed without penalties. In the perpendicular parking task, the training success rate was strongly dependent on the initial conditions—particularly whether an appropriate distance to the parking space was maintained—and averaged around 40%. However, in the official competition the initial conditions were not satisfied, resulting in failure of the parking mission.

6.CONCLUSION

This study presented the design and implementation of a modular autonomous driving software framework based on ROS2, which was applied and validated in the Sungkyunkwan University Autonomous Driving Competition. The proposed framework was composed of independent camera, LiDAR, decision-making, and control modules, allowing flexible combinations tailored to each task. This modularity reduced unnecessary computations, improved debugging efficiency, and enabled rapid reconfiguration according to competition requirements.

In the competition results, the general driving task demonstrated stable lane keeping and achieved a successful completion time. The mission driving task showed reliable performance in static obstacle avoidance and traffic light recognition, while highlighting limitations in handling dynamic obstacles. For the perpendicular parking task, the framework exhibited high sensitivity to initial conditions,

which led to a failure in the final run but also provided valuable insight into system weaknesses.

Overall, the results confirm that a ROS2-based modular framework can effectively address diverse driving scenarios. Future work will focus on improving robustness against varying initial conditions and enhancing perception and control performance in dynamic environments to ensure more reliable operation in real-world applications.

References

[1] Ultralytics, "Segmentation," Ultralytics YOLOv11 Docs. [Online]. Available: https://docs.ultralytics.com/tasks/segment/. [Accessed: Sep. 8, 2025].

[2] Ultralytics, "Ultralytics YOLO," GitHub, ultralytics/ultralytics. [Online]. Available: https://github.com/ultralytics/ultralytics. [Accessed: Sep. 8, 2025].

[3] J. Lee, et al., "Implementation of Control Systems and Algorithms for Autonomous Driving Vehicles," in Proc. KSAE Annual Autumn Conf., pp. 2948-2954, Nov. 2024.

[4] M. Oh, Y. Shin, J. Oh, and H. Moon, "Implementation of Lane Detection Based Autonomous Driving with YOLOv8 and Simultaneous Lane Correction Algorithm through Steering Angle, Pixel Referencing," in Proc. KSAE Annual Autumn Conf., pp. 2906-2910, Nov. 2024.

[5] J. Hong, W. Yoon, and N. Kwon, "Vision-based Lane Detection and Steering Decision Algorithm for Autonomous Driving," in Proc. IEIE Summer Annual Conf., pp. 1011-1015, Jun. 2023.

[6] H. Hong, J. Lee, S. Lee, and J. W. Jeon, *ROS2-based Autonomous Vehicle Design and Implementation*. Seoul, Korea: Sungkyunkwan University Press, 2025.

[7] Y. Kim, K. Kwon, H. Hong, S. Lim, and J. Jeon, "Improved Autonomous Driving Algorithm for Multi-Obstacle Avoidance," in Proc. KSAE Annual Autumn Conf., pp. 2971-2974, Nov. 2024.

[8] S. Bae, S. Choi, and M. Chung, "Improving Obstacle Avoidance Decision Algorithm Using YOLOv8," in Proc. KSAE Annual Autumn Conf., pp. 2986-2989, Nov. 2024.

[9] H. Park, S. Lee, S. Lee, Y. Nam, H. Hong and J. Jeon. "Lane Detection Improvement Based on Deep Learning Model." Annual Conference of KIPS, vol. 31, no. 1, pp. 650-651, May 2024.

[10] Sungkyunkwan University, "The 3rd Sungkyunkwan University Autonomous Driving Competition," YouTube, [Online]. Available: https://www.youtube.com/live/r8NrDGU_-ZI?feature=shared. [Accessed: Sep. 9, 2025].

Acknowledgements

This work was supported by the Korea Institute for Advancement of Technology (KIAT) grant funded by the Ministry of Education (Project No. P0030064, Advanced Industry Talent Training Bootcamp Program – Sungkyunkwan University (Future Mobility)).